

ScamGram — Technical Guide for Questions 3 to 6

Project Title: AI-Driven Spam Detection and Scam Engagement

This document explains in detail how the ScamGram system implements Questions 3 through 6 of the MSc Semester Project. Each section references the exact source files and code lines involved. The system is built in Python using Flask for the web interface, scikit-learn for machine learning classification, and the Groq API (LLaMA 3.3) for generative AI reply generation.

Project Architecture Overview

The project consists of the following key files:

File	Purpose
Q2/preprocess_question2.py	Preprocessing pipeline — extracts 39 numeric features from raw email text (Question 2)
Q2/datasets/	Folder containing the raw CSV email datasets (~2,500 emails)
Q2/training_dataset.csv	Output of the preprocessing pipeline — the feature-enriched training CSV
train_classifier.py	Trains the SVM classifier on the preprocessed dataset and saves the model
spam_model.pkl	The trained SVM model (serialized with pickle)
scaler.pkl	The StandardScaler used to normalize features (serialized with pickle)
script.py	Main application — classification, reply generation, email handling, Flask GUI server
templates/gui.html	The web-based GUI (Telegram-style chat interface)
scambait_history.json	Persistent JSON file storing all scambaiting conversation threads

How the Files Connect

The workflow follows a clear pipeline:

```
Raw CSVs (Q2/datasets/)
  → preprocess_question2.py extracts 39 features
    → training_dataset.csv
      → train_classifier.py trains SVM
        → spam_model.pkl + scaler.pkl
          → script.py loads model and classifies live emails
            → Groq API generates scambaiting replies
              → gui.html displays everything in real-time
```

Question 3 — Review of Related Work

"Review related work on spam detection, phishing classification, and AI-assisted email filtering in order to justify the chosen method."

3.1 Spam Detection

The management and identification of unsolicited electronic correspondence, commonly referred to as spam, represents one of the oldest and most persistent challenges in the field of information systems security. In the initial phases of developing relevant defensive mechanisms, systems relied almost exclusively on static rule-based filtering and the deployment of global exclusion or approval registries, known as blacklists and whitelists respectively. These approaches, while presenting low computational overhead, quickly proved inadequate as they depended on identifying specific Internet Protocol (IP) addresses or rigid alphanumeric patterns. Spam originators could easily bypass these barriers by continuously altering their infrastructure and introducing minimal obfuscating variations to the textual content. The imperative for more dynamic, adaptive systems led academic literature to adopt Machine Learning (ML) paradigms, which enabled the statistical analysis of message characteristics rather than reliance on fixed signatures.

At the level of conventional Machine Learning models, text classification was historically built upon vector space representations, such as the Bag-of-Words model and Term Frequency-Inverse Document Frequency (TF-IDF) statistical frameworks. Operating on these mathematical representations, algorithms such as Naive Bayes which is fundamentally grounded in the eponymous probabilistic theorem Support Vector Machines (SVM), and Random Forests constituted the core of academic research for decades. As established by comprehensive comparative studies of text classifiers (Mehndiratta et al., 2018), these algorithms exhibit exceptional efficiency in detecting traditional, bulk spam distributions. This efficiency stems from their ability to isolate statistically significant deviations in the frequency of specific keywords and lexical tokens associated with commercial or fraudulent promotions.

However, the rapid evolution of spam generation tactics has exposed the structural limitations of these classical architectures. The primary vulnerability of traditional Machine Learning algorithms lies in their fundamental inability to comprehend semantic context, syntactical dependencies, and the broader thematic awareness of natural language. Because these algorithms treat textual strings as unstructured collections of independent words, they consistently fail to discern the subtle intent distinguishing a legitimate promotional email dispatched by an authorized subscription service from a malicious spam campaign. Consequently, this limitation yields high rates of false positives, resulting in the erroneous quarantine of critical user correspondence. Furthermore, modern email provider architectures (Google, 2024) enforce stringent authentication frameworks including Sender Policy Framework (SPF), DomainKeys Identified Mail (DKIM), and Domain-based Message Authentication, Reporting, and Conformance (DMARC) which have forced adversaries to abandon overt, mass-distributed spam in favor of highly sophisticated, personalized narratives that closely mirror legitimate human interaction.

3.2 Phishing Classification

The transition from conventional spam to phishing campaigns represents a qualitative escalation in cyber threats. Phishing does not merely aim to cause user inconvenience or advertise illicit products; instead, it involves deliberate deception designed to exfiltrate sensitive data, including authentication credentials, financial information, and personally identifiable information (PII). As documented in cybersecurity technical

reports, these attacks have evolved from mass, random dispatches into highly targeted Social Engineering operations. Notable variants include Spear Phishing, which targets specific personnel within an organizational structure, and Whaling, which exclusively targets high-level corporate executives. Attackers leverage advanced techniques to counterfeit the visual corporate identity and linguistic style of trusted institutions, such as banking corporations, government agencies, or global digital platforms, making visual detection by the end-user exceedingly difficult.

Modern academic research addressing this phenomenon focuses heavily on the development of hybrid classification systems that evaluate incoming messages across multiple independent dimensions. According to systematic literature reviews (Ramanathan et al., 2024), the parametric axes of analysis are broadly divided into the structural characteristics of the message and its semantic substance. Structural analysis encompasses the programmatic verification of electronic mail headers, the cryptographic validation of domain authenticity, and the deployment of machine learning models to assess embedded Uniform Resource Locators (URLs). The utilization of public datasets for model training has demonstrated that combining features such as redirection chains, URL character lengths, and the presence of anomalous punctuation can yield robust detection rates against established attack vectors.

Nevertheless, the critical vulnerability in automated defense remains the analysis of the email body itself, where adversaries exploit human psychological vulnerabilities. To counteract this, computer science has pivoted toward advanced Natural Language Processing (NLP) techniques and Deep Learning architectures, such as Long Short-Term Memory (LSTM) recurrent networks and Transformer-based language models, most notably Bidirectional Encoder Representations from Transformers (BERT). Experimental data demonstrates that these deep models significantly outperform traditional statistical approaches because they can model semantic intent (Ravi and Chalil, 2018). Deep neural networks can successfully detect the artificial cultivation of urgency, the presence of veiled coercion, or the systemic pressure for immediate compliance elements that constitute the defining markers of social engineering exploits.

3.3 AI-assisted Email Filtering

The integration of Generative Artificial Intelligence (Generative AI) and Large Language Models (LLMs) has initiated a radical paradigm shift in the architectural design of email filtering systems. Modern enterprise mail environments have progressed far beyond binary classification models that simply segregate incoming streams into ham or spam. Instead, next-generation infrastructures function as integrated, intelligent AI Assistants capable of performing deep, dynamic analysis on information flows in real time. These advanced systems are not restricted to passive threat detection; they execute granular categorization of messages based on stylistic tone, generate abstract summaries of complex corporate threads, and autonomously prioritize mail flows based on perceived contextual importance.

This technological evolution shifts the defensive perimeter of information systems away from the traditional analysis of known signatures and lexical indicators toward a comprehensive behavioral and semantic analysis framework. As examined in current literature (Sallam et al., 2023), Large Language Models possess a unique capacity for zero-shot generalization, enabling them to counter zero-day phishing attacks novel, emerging attack mutations that have never been logged in any historical threat database and share no structural similarities with prior exploits. The comprehension of subtle conceptual nuances allows AI models to perceive the underlying malicious nature of a message even when it is written in an entirely polite, professional register and lacks overt indicators such as malicious URLs or suspicious file attachments, which is typical of advanced Business Email Compromise (BEC) operations.

3.4 Justification of the Chosen Methodology for Questions 4 & 5

For the practical implementation of the proposed defensive solution, this project adopts a **hybrid architecture** that combines the efficiency and interpretability of conventional Machine Learning with the generative capabilities of Large Language Models (LLMs). Specifically, the system employs a locally trained **Support Vector Machine (SVM)** as the primary classification engine, while leveraging the **Groq API** (deploying Llama 3.1 and Llama 3.3 models) for two complementary purposes: fallback classification resilience and automated scambaiting reply generation. This dual-layer design is grounded in the conclusions of the preceding literature review and is technically justified across the following four dimensions:

Efficient and Interpretable Classification via SVM with Handcrafted Features: The core classification engine is a Support Vector Machine with an RBF (Radial Basis Function) kernel, trained on 39 handcrafted features extracted from email body text by the Q2 preprocessing pipeline (`Q2/preprocess_question2.py`). These features were specifically designed to capture the structural and lexical fingerprints of advance-fee fraud and phishing emails, including urgency language frequency, financial term density, bank detail request patterns, and advance fee indicators. As established in the literature (Mehndiratta et al., 2018), SVMs exhibit exceptional efficiency in high-dimensional feature spaces with relatively small datasets. By combining domain-expert feature engineering with a proven statistical classifier, the system achieves high accuracy while maintaining full transparency: every classification decision can be traced back to specific, interpretable feature values. The SVM also provides calibrated probability estimates via `predict_proba()`, enabling the system to display a confidence percentage (e.g., 98.5%) alongside each classification, giving the operator quantitative visibility into the model's certainty.

LLM-based Fallback for Classification Resilience: While the SVM serves as the primary classifier, the architecture incorporates a graceful fallback mechanism. If the pre-trained model files (`spam_model.pkl`, `scaler.pkl`) fail to load due to file corruption, environment migration, or initial deployment before training, the system automatically delegates classification to the Llama-3.1-8b-instant model via the Groq API (`script.py`, lines 182–219). This fallback classifier uses a detailed system prompt that instructs the LLM to analyze the email and return a structured JSON classification with confidence score and reasoning. The prompt specifies a minimum 85% confidence threshold before labeling a message as SPAM, reducing false positives. This design ensures the system remains fully operational even when local model artifacts are unavailable, combining the speed of local inference with the resilience of cloud-based AI.

Active Defense Capabilities via Generative Scambaiting: A core innovation of this project is the transition from passive, reactive filtering to active defense through automated adversarial deception, known as scambaiting. Traditional classification models (such as SVMs or feedforward neural networks) are structurally incapable of text generation. By leveraging the Llama-3.3-70b-versatile model through the Groq API (`script.py`, lines 223–284), the system synthesizes dynamic, contextually coherent responses directed back at fraudulent senders. The generated replies are crafted through a specialized system prompt that instructs the LLM to impersonate an enthusiastic, gullible recipient ("Alex Johnson") who asks for extensive clarifying details — wasting the attacker's operational resources and time while completely masking the user's authentic identity. Critically, all generated replies pass through a **human-in-the-loop approval workflow** before transmission, ensuring that no automated response is sent without explicit operator authorization.

Speed, Scalability, and Infrastructure Optimization: The hybrid architecture optimizes infrastructure costs by reserving the computationally expensive LLM calls exclusively for tasks that require generative capabilities (reply generation and fallback classification), while routing the high-frequency classification workload through the lightweight, locally executed SVM model. When LLM access is needed, the hardware-accelerated

architecture of the Groq API provides elite token-per-second processing velocities, enabling generative response cycles to execute in near real time. This serverless API approach completely absolves the organization from the capital expenditure and maintenance overhead associated with provisioning high-end local GPUs required to execute large-scale deep learning models.

In conclusion, the proposed methodology which unifies **local SVM classification** for fast, interpretable spam detection with **LLM-powered generative scambaiting** for active adversarial engagement represents a practical, dual-layer defense architecture. It successfully shifts the role of artificial intelligence from a passive boundary filter into an autonomous, active cyberdefense mechanism, while maintaining operational resilience through its fallback classification layer.

Question 4 — AI/ML Classification Model

"Design, train, or integrate an AI/ML model that classifies emails as legitimate or spam. Explain the selected approach, feature choices, and evaluation method."

4.1 Selected Approach & Architecture

For the spam and scam email classification system in `script.py`, a **hybrid dual-layer architecture** was implemented that combines a locally trained Machine Learning classifier with an API-driven Large Language Model (LLM) fallback. The **primary classification engine** is a Support Vector Machine (SVM) with an RBF kernel, trained on 39 handcrafted features extracted by the Q2 preprocessing pipeline (`Q2/preprocess_question2.py`). This model is loaded at startup from serialized files (`spam_model.pkl` and `scaler.pkl`) and executes classification locally with no network dependency (`script.py`, lines 35–45).

As a **secondary fallback layer**, the system utilizes the `llama-3.1-8b-instant` model hosted on the Groq inference engine via the Groq Python client. This fallback is activated **only** when the local SVM model files are unavailable (e.g., due to file corruption, environment migration, or initial deployment before training). When active, the LLM exploits its deep semantic understanding and zero-shot reasoning capabilities to perform classification. By supplying an engineered system prompt, the LLM is instructed to operate as a high-precision binary classifier. To guarantee deterministic outputs and facilitate seamless programmatic integration, the API call is configured with a temperature of 0.0 and utilizes JSON mode (`response_format={"type": "json_object"}`). The model returns a structured JSON payload containing the classification result (SPAM or LEGITIMATE), the evaluation confidence (0–100%), and a logical reasoning justification (`script.py`, lines 182–219).

This hybrid design ensures maximum operational resilience: the system maintains full functionality regardless of whether the local model artifacts are present or not.

4.2 Feature Selection & Preprocessing

The SVM classifier operates on a carefully engineered feature vector of **39 numeric features** extracted by the `extract_email_features()` function from the Q2 preprocessing pipeline (`Q2/preprocess_question2.py`, lines 218–339). These features are specifically designed to capture the structural and lexical fingerprints of advance-fee fraud and phishing emails, and are grouped into four categories:

Basic Text Statistics (14 features): Structural metrics including character count, word count, sentence count, paragraph count, line count, average word length, average sentence length, uppercase ratio, digit ratio,

punctuation count, exclamation count, question count, multiple exclamation presence, and all-caps word count.

Entity Detection (5 features): Regular expression-based detection of URLs, email addresses, phone numbers, monetary expressions (e.g., "\$1,000,000", "5 million"), and anonymized entity tokens (e.g., `PERSON_TOKEN`, `ORG_TOKEN`).

Keyword Category Counts (12 features): Frequency counts of predefined keyword lists characteristic of scam emails, including urgency terms ("urgent", "immediately"), secrecy terms ("confidential", "secret"), financial terms ("million", "inheritance"), bank terms ("IBAN", "wire transfer"), personal information requests ("passport", "full name"), action requests ("send", "reply", "click"), fee terms ("processing fee", "customs"), greeting terms ("dear", "beloved"), signature terms ("regards", "sincerely"), title terms ("prince", "barrister"), country terms ("nigeria", "ghana"), and organization terms ("ministry", "embassy").

Structural Patterns (8 features): Composite boolean features that detect specific scam patterns by combining multiple keyword categories. For example, `has_advance_fee_pattern` fires when fee terms co-occur with payment action terms, and `has_bank_details_request_pattern` fires when bank terms co-occur with action verbs like "send" or "provide".

At runtime, the `classify_email()` function in `script.py` (line 150) calls `extract_email_features(body)` on the incoming email body, constructs a DataFrame aligned with the exact column order used during training, normalizes the feature vector using the saved `StandardScaler`, and passes it to the SVM's `predict()` method (lines 142–178).

Additionally, to maintain conversation-level context in multi-turn scambaiting scenarios, the system extracts a key identifier (`conv_key`) composed of the sender's reply address and a normalized subject line (stripped of common reply/forward prefixes like Re:, Fwd:, and απ: via the `normalize_subject()` function, lines 74–86). If this key matches a record in the local database (`scambait_history.json`), the message is automatically bypass-classified as SPAM with 100% confidence, reinforcing temporal feature consistency (`script.py`, lines 564–571).

4.3 Training Process & Evaluation

The SVM model is trained by the `train_classifier.py` script using the preprocessed dataset produced by Q2:

1. **Dataset Loading:** The training data is loaded from `Q2/training_dataset.csv`, which contains approximately 2,500 labeled emails with their 39 extracted features and a binary label (`flag_binary`: 1 = spam, 0 = legitimate).
2. **Train/Test Split:** The dataset is split 80/20 with stratification to maintain class balance (`train_test_split` with `stratify=y`, line 36).
3. **Feature Normalization:** A `StandardScaler` is fitted on the training set to ensure all 39 features have zero mean and unit variance, preventing features with larger scales from dominating the SVM's decision boundary (lines 39–41).
4. **SVM Training:** The classifier is trained using `SVC(kernel='rbf', probability=True, random_state=42)` (lines 45–46). The RBF kernel enables non-linear decision boundaries, and `probability=True` enables calibrated probability estimates via Platt scaling.

5. **Evaluation:** The model is evaluated on the held-out test set using `accuracy_score` and a full `classification_report` that provides precision, recall, and F1-score for both classes (lines 49–56).
6. **Model Persistence:** The trained SVM and scaler are serialized to `spam_model.pkl` and `scaler.pkl` using pickle (lines 59–66).

For the Groq fallback classifier, the evaluation framework uses a **conservative 85% confidence threshold** within the system prompt instructions. The model is directed to only designate an email as SPAM if its confidence rating exceeds this threshold. In cases of ambiguity, the default fallback is to classify the email as LEGITIMATE. This decision rule ensures that the automatic scambaiting responder is only triggered on highly certain phishing or scam vectors. Additionally, the classification performance and runtime behavior are audited via a terminal execution report that updates metrics such as total processed emails, identified spam, legitimate communications, and executed replies, allowing for empirical monitoring of filtering efficiency (`script.py`, lines 644–653).

Question 5 — Generative AI Automatic Replies

"Leverage a generative AI model, such as ChatGPT, Claude, or another comparable system, to produce automatic replies to simulated scam emails."

5.1 Model Selection and Configuration

To transition from passive filtering to active engagement, the system employs a secondary Generative AI model to autonomously draft responses to identified scam emails. As implemented in the `generate_scam_reply()` function (`script.py`, lines 223–284), the system leverages the `llama-3.3-70b-versatile` model via the Groq API. This larger parameter model was specifically selected for its advanced natural language generation capabilities, allowing it to produce highly contextual and grammatically nuanced replies. Unlike the deterministic classifier (which operates at a temperature of 0.0), the generative endpoint is deliberately configured with a temperature of 0.7 (line 280). This introduces creative variance, ensuring that the generated replies appear organic and human-like, thereby preventing scammers from realizing they are interacting with an automated script.

5.2 Prompt Engineering and Persona Design

The core mechanism driving the automated engagement is a meticulously engineered system prompt (`script.py`, lines 231–245) designed to execute the scambaiting strategy. The LLM is instructed to adopt the persona of a cautious but interested potential victim named "Alex Johnson". To effectively waste the scammer's time, the prompt explicitly directs the model to express enthusiasm while simultaneously demanding granular, clarifying details — such as official company registration numbers, physical addresses, and direct phone lines. The prompt also instructs the model to write in the same language as the original scam email, enabling it to respond appropriately to scams received in any language (including Greek). Crucially, the system prompt acts as a security sandbox; it contains strict negative constraints prohibiting the model from divulging real Personally Identifiable Information (PII) or revealing that it is an AI, ensuring a secure and controlled interaction.

For first-turn replies, the function constructs a user prompt containing the sender address, subject, and the first 1,500 characters of the body text (lines 262–268):

```
user_prompt = (  
    "Original scam email:\n"  
    "From: " + sender + "\n"  
    "Subject: " + subject + "\n\n"  
    "Body:\n" + body[:1500] + "\n\n"  
    "Write the reply now:"  
)
```

5.3 Human-in-the-Loop Approval Workflow

A critical safety feature of the system is that generated replies are **never sent automatically**. Instead, they are stored in the conversation history (`scambait_history.json`) with a `pending_approval` status (`script.py`, lines 612–616):

```
history[conv_key].append({  
    "role": "assistant",  
    "content": reply,  
    "status": "pending_approval"  
})
```

The operator must explicitly review and approve each draft reply through the web GUI before it is transmitted. This **human-in-the-loop** design provides a critical safety layer that:

- Prevents unintended email sending to legitimate senders (false positive protection)
- Allows the operator to review and potentially edit generated content before transmission
- Ensures compliance with ethical guidelines by maintaining human oversight over all automated communications
- Provides an opportunity to reject and delete false positive conversations entirely

5.4 Execution and Delivery Mechanism

When the operator approves a reply through the GUI, the `/api/approve_reply` endpoint (`script.py`, lines 713–756) handles the delivery process:

1. The pending reply is located in the conversation history
2. The recipient address and subject are parsed from the conversation key
3. The reply text is dynamically injected into a MIME multipart object using the `send_reply()` function, which establishes a secure SMTP connection over STARTTLS (`smtp.gmail.com:587`) to transmit the fabricated reply directly back to the scammer's extracted Reply-To address
4. The message status is updated from `pending_approval` to `sent` in the history file

This seamless handoff from the ML classifier to the Generative AI responder, mediated by human approval, creates a supervised defensive mechanism that actively disrupts fraudulent campaigns while maintaining full operator control.

Question 6 — Multi-Turn Conversation Behavior

"Demonstrate multi-turn behavior by showing how the system can continue responding when additional scam messages are received."

6.1 State Management and Conversation Tracking

To effectively engage scammers over prolonged periods, the system was engineered to support multi-turn conversational behavior, moving beyond stateless, single-reply interactions. This requires robust state management to track ongoing threads. The script achieves this by generating a unique session identifier (`conv_key`) for every incoming email, formulated by concatenating the sender's extracted Reply-To address with a normalized version of the email's subject line (`script.py`, line 563):

```
norm_subj = normalize_subject(subject)
conv_key = f"{reply_addr} | {norm_subj}"
```

The `normalize_subject()` function (lines 74–86) strips repetitive prefixes such as "Re:", "Fwd:", "Fw:", and "απ:" (Greek reply prefix), and converts the subject to lowercase. This normalization ensures that all replies within a thread map to the same conversation key, regardless of how many times the subject has been prefixed by email clients. This composite key allows the system to map incoming messages to existing conversation arrays stored locally in a persistent JSON database (`scambait_history.json`).

6.2 Context Window Integration and Classifier Bypass

When a new email arrives that matches an active session key (`is_ongoing_scambait`), the system initiates a strategic bypass (`script.py`, lines 566–571). Recognizing the email as part of an ongoing engagement, it overrides the initial classification phase, automatically tagging the message as SPAM with 100% confidence:

```
classification = "SPAM"
confidence = "100% (Ongoing)"
scam_prob = 1.0
reason = "Existing active scambaiting conversation thread."
```

6.3 Quoted Reply Stripping

Before storing the scammer's new message, the system applies the `strip_quoted_reply()` function (`script.py`, lines 89–111) to remove quoted reply text that email clients typically append. When a scammer replies, their email usually contains the full text of our previous message prefixed with `On <date> <sender> wrote:` followed by lines starting with `>`. The function handles two common quoting styles:

1. **Gmail/Outlook style:** Detects the `On ... wrote:` marker and removes everything from that point onwards
2. **Generic quoting:** Removes consecutive lines starting with `>` (standard email quoting convention)

This cleaning ensures that only the scammer's **new, original content** is stored in the conversation history (`script.py`, line 592):

```
clean_body = strip_quoted_reply(body[:2000])
```

6.4 Context-Aware Reply Generation

To generate the subsequent reply, the system retrieves the entire conversation history (comprising all previous scammer messages and the system's generated replies) and appends the newest message. This concatenated array is passed to the LLM's context window (`script.py`, line 259):

```
messages = [{"role": "system", "content": system_prompt}] + cleaned_history
```

Before passing the history to the Groq API, the system performs a critical cleaning step (lines 251–258). Each message is stripped of metadata fields (`scam_probability`, `status`) that are used internally but would cause API errors if included. Draft messages with `pending_approval` status are also excluded to avoid sending incomplete context:

```
cleaned_history = []
for msg in conversation_history:
    if msg.get("role") == "assistant" and msg.get("status") == "pending_approval":
        continue
    cleaned_history.append({
        "role": msg["role"],
        "content": msg["content"]
    })
```

Consequently, the `llama-3.3-70b-versatile` model processes the full temporal sequence of the interaction, enabling it to reference past statements, answer specific follow-up questions, and dynamically escalate its delay tactics across multiple exchanges. For example:

- If the scammer mentioned a bank transfer in turn 1, the AI might ask for the bank name in turn 2
- If the scammer provided a reference number, the AI remembers it and uses it in subsequent replies
- The conversation evolves naturally, keeping the scammer engaged for as long as possible

6.5 Persistent Storage

All conversation data is saved to `scambait_history.json` after every change (`script.py`, line 617):

```
save_history(history)
```

This JSON file stores the complete history of all scambaiting sessions, including:

- Each message's role (`user` for scammer, `assistant` for our replies)
- The message content (cleaned of quoted text for scammer messages)
- The spam probability score (for scammer messages, from SVM's `predict_proba()`)

- The approval status (for our replies: `pending_approval` or `sent`)

This persistence ensures that conversations survive server restarts and can continue indefinitely, supporting arbitrarily long multi-turn engagements.

Web GUI — ScamGram Dashboard

The web interface (`templates/gui.html`) provides a real-time monitoring dashboard inspired by the Telegram messaging app. It is served by a Flask web server embedded in `script.py` (lines 656–825).

Interface Layout

The GUI is split into two main panels:

- **Left Sidebar:** Shows a list of all active scambaiting conversations. Each entry displays the scammer's email address, the email subject, a spam probability badge (e.g., "98.5%"), and a message count. A search bar allows filtering conversations.
- **Right Chat Panel:** When a conversation is selected, this panel displays the full message thread in a chat-bubble format similar to Telegram. Scammer messages appear on the left side with a red sender label, while our system replies ("Alex Johnson") appear on the right side with a green sender label.

Key Features

1. **Live Email Check:** A "Check Email Live" button triggers a background scan of the Gmail inbox. The system connects via IMAP, retrieves unread emails, classifies them, and generates replies — all in real-time.
2. **Approval System:** Draft replies appear with a dashed orange border and a "DRAFT — PENDING APPROVAL" badge. The user sees two action buttons at the bottom:
 - **Approve & Send:** Sends the reply via SMTP and marks it as sent
 - **Reject Draft:** Discards the generated reply without sending
 - **Delete Thread:** Removes the entire conversation (useful for false positives)
3. **Auto-Refreshing History:** The conversation history refreshes automatically every 5 seconds, so new emails and generated replies appear without manual page reloads.
4. **Spam Probability Display:** Each scammer message shows the SVM model's spam probability as a percentage badge, giving the user visibility into the classifier's confidence.
5. **Search Functionality:** The sidebar includes a search bar that filters conversations by scammer name, email address, or subject line in real-time.

Technical Implementation

The GUI is a single-page HTML application using vanilla JavaScript (no framework). Key API endpoints:

Endpoint	Method	Purpose
<code>/api/history</code>	GET	Returns all conversation data from <code>scambait_history.json</code>

Endpoint	Method	Purpose
/api/check_mail	POST	Triggers a background email check thread
/api/check_status	GET	Returns whether an email check is currently running
/api/approve_reply	POST	Sends the approved reply via SMTP
/api/reject_reply	POST	Discards a draft reply or deletes a conversation

The Flask server runs locally on `http://127.0.0.1:5000/` and opens the browser automatically on startup (lines 820–822).

Design Choices

The interface uses a dark theme with the following color palette:

- Background: Dark navy (`#0e1621`)
- Sidebar: Slightly lighter navy (`#17212b`)
- Accent: Blue (`#5288c1`)
- Scam alerts: Red (`#ef4444`)
- System status: Green (`#10b981`)
- Pending/Draft: Orange (`#f59e0b`)

Typography uses the **Inter** font family (Google Fonts) for a clean, modern appearance. Icons are provided by the **Lucide** icon library.

Summary of Implementation Mapping

Project Question	Implementation	Key Files & Lines
Q2 — Preprocessing pipeline	Feature extraction (39 features)	<code>Q2/preprocess_question2.py</code> (lines 218–339)
Q3 — Related work review	Theoretical — justifies SVM + Groq choices	Written report
Q4 — ML classification model	SVM with RBF kernel trained on 39 features	<code>train_classifier.py</code> (full file), <code>script.py</code> (lines 134–219)
Q5 — Generative AI replies	Groq API (LLaMA 3.3) with scambaiting prompt	<code>script.py</code> (lines 223–284)
Q6 — Multi-turn behavior	Conversation history + context-aware generation	<code>script.py</code> (lines 561–617, 247–259)
GUI — Web dashboard	Telegram-style Flask app with approval workflow	<code>templates/gui.html</code> , <code>script.py</code> (lines 656–825)